

# MACHINE LEARNING IN FINANCE

## MODULE 6 | LESSON 4

---

# CREDIT RISK MODELING: APPLICATION OF DEEP LEARNING

---

|                        |                                                                 |
|------------------------|-----------------------------------------------------------------|
| <b>Reading Time</b>    | 120 minutes                                                     |
| <b>Prior Knowledge</b> | Introduction to machine learning, introduction to deep learning |
| <b>Keywords</b>        | Expected Loss, credit risk, SMOTE                               |

---

*In the previous lessons of this module, we have built on the theory of deep learning. In this lesson, we will discuss one of the hottest topics in machine learning, that is, predicting loan default. We will use classical machine learning algorithms on a refined dataset and discuss the steps used to predict mortgage default.*

## 1. Mortgage Delinquency

There has been an evolution in the business of lending money as the process has become increasingly complex due to the growing market demands and clients' increasing appetite for credit. These factors among others have led to an increase in regulation and oversight in the banking industry so as to make sure they act responsibly when issuing loans. In the recent past, the rate of digitalization globally has shot up with people in remote parts of the world having access to phones. This has made it possible for people to use mobile devices as a financial medium through which they can send and receive money to and from other people around the world. These transactions happen in a matter of seconds. Many fintechs have taken advantage of this to launch microloans to customers who are low risk. The fintechs use the interaction of the customers with their gadgets to build a credit score for each of the customers and determine the probability of the customer defaulting on a loan. This logic also applies to mortgages and the probability of mortgage default is called mortgage delinquency. Machine learning models are trained on the data we have processed and the decision making process of giving loans is automated.

The machine learning models are used to assess the creditworthiness of a borrower. Before the advent of machine learning, lenders had an established guideline to measure creditworthiness. These guidelines were based on the five C's listed below:

1. Character that looks at the borrower's repayment and credit record.

2. Capacity that assess the borrower's ability to service the loan by looking at the debt-to-income ratio.
3. Capital that looks at the down payment the borrower has paid. This is used to determine how serious the borrower is.
4. Collateral, which is the asset provided to secure the mortgage, such as another home.
5. Conditions of the borrower's environment, like the state of the economy.

However, this has posed serious challenges to lenders as the number of features are limited in assessing customers' creditworthiness, with potentially credit-worthy clients being denied credit for failing certain criteria, and their inability to keep pace with the technological evolution that has been witnessed in the past decade.

It is because of these limitations that machine learning models are now at the heart of assessing the creditworthiness of borrowers. However, recent research has shown that deep learning has the potential to eclipse machine learning for assessing credit risks. Deep neural networks are great at detecting risky borrowers when the data is unstructured and very complex.

However, the risk of using deep learning models is that in most cases they are not explainable, that is, they are like black boxes and we are in most cases unable to know what happened for us to get a certain output. Currently, a lot of research is being done to address this by focusing on explainable artificial intelligence. In the next section, we are going to show a simple example of using a classical machine learning algorithm to assess credit risk.

## 2.Credit Risk

### 2.1 Data Sources

We downloaded our data from the [UCI marketing](#) dataset, and we did some engineering and cleaning to make the data ready for model training.

In the cell below, we import packages we will use in this demonstration.

```
In [2]: # !pip install imbalanced-Learn
```

```
In [3]: from collections import Counter

# EDA
import matplotlib.pyplot as plt
import numpy as np

# data manipulation
import pandas as pd
import seaborn as sns
from imblearn.combine import SMOTETomek
from scipy import stats

# feature selection
```

```

from sklearn.ensemble import RandomForestClassifier

# algorithms
from sklearn.linear_model import LogisticRegression

# model evaluation
from sklearn.metrics import (
    accuracy_score,
    brier_score_loss,
    classification_report,
    cohen_kappa_score,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)

# machine learning
from sklearn.model_selection import train_test_split

```

## 2.2 Loading the Dataset

We start by uploading this downloaded data and taking a rough look at the data. In this problem, we will skip the process of performing EDA and data preprocessing techniques as the data is already processed.

- Person Age, Person Income, Person Employment Length, Loan Amount, Loan interest rate, Loan percent income and Person credit history length were transformed using min max scaler, that is, converting the values to range between 0 and 1.
- The other columns were categorical and we therefore converted them to numerical data by one hot encoding the columns.

```
In [4]: df = pd.read_csv("credit_sample.csv")
```

```
In [5]: df.head()
```

```
Out[5]:
```

|   | loan_status | person_age | person_income | person_emp_length | loan_amnt | loan_int_rate |
|---|-------------|------------|---------------|-------------------|-----------|---------------|
| 0 | 0           | 0.015625   | 0.002751      | 0.121951          | 0.014493  | 0.32134       |
| 1 | 1           | 0.078125   | 0.002751      | 0.024390          | 0.144928  | 0.41853       |
| 2 | 1           | 0.046875   | 0.030209      | 0.097561          | 1.000000  | 0.55112       |
| 3 | 1           | 0.062500   | 0.024757      | 0.195122          | 1.000000  | 0.49719       |
| 4 | 1           | 0.015625   | 0.002898      | 0.048780          | 0.057971  | 0.09662       |

5 rows × 7 columns



```
In [6]: df.shape
```

```
Out[6]: (29459, 27)
```

```
In [7]: df.dtypes
```

```
Out[7]: loan_status          int64
person_age          float64
person_income        float64
person_emp_length    float64
loan_amnt            float64
loan_int_rate         float64
loan_percent_income   float64
cb_person_cred_hist_length float64
person_home_ownership_MORTGAGE int64
person_home_ownership_OTHER int64
person_home_ownership_OWN int64
person_home_ownership_RENT int64
loan_intent_DEBTCONSOLIDATION int64
loan_intent_EDUCATION int64
loan_intent_HOMEIMPROVEMENT int64
loan_intent_MEDICAL int64
loan_intent_PERSONAL int64
loan_intent_VENTURE int64
loan_grade_A int64
loan_grade_B int64
loan_grade_C int64
loan_grade_D int64
loan_grade_E int64
loan_grade_F int64
loan_grade_G int64
cb_person_default_on_file_N int64
cb_person_default_on_file_Y int64
dtype: object
```

```
In [8]: df.loan_status.value_counts()
```

```
Out[8]: loan_status
0      22996
1       6463
Name: count, dtype: int64
```

Clearly, the data are highly imbalanced.

```
In [9]: # plots count
ax = sns.countplot(x=df["loan_status"])

# sets the figure size in inches
ax.figure.set_size_inches(12, 6)

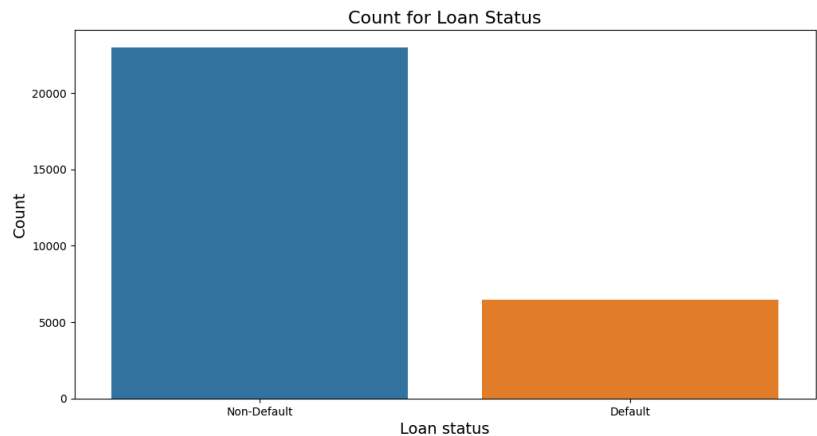
# set plot features
ax.set_title("Count for Loan Status", fontsize=16)
ax.set_ylabel("Count", fontsize=14)
ax.set_xlabel("Loan status", fontsize=14)

# set `xticks` labels
plt.xticks([0, 1], ["Non-Default", "Default"])

plt.suptitle(
    "Fig. 1: Top: A bar plot of good versus bad loans. The graph shows that the
    fontweight='bold",
    horizontalalignment="right",
)
```

```
# displays plot
plt.show()
```

Fig. 1: Top: A bar plot of good versus bad loans. The graph shows that the dataset is highly imbalanced.



```
In [10]: # separating the data set for easier analysis
df_default = df[df["loan_status"] == 1].copy()
df_non_default = df[df["loan_status"] == 0].copy()

# counts the number of defaults and non-defaults
total_default = df_default.shape[0]
total_non_default = df_non_default.shape[0]
total_loans = df.shape[0]

print("Number of default cases:", total_default)
print(
    "This is equivalent to {:.2f}% of the total loans".format(
        (total_default / total_loans) * 100
    )
)

print("\nNumber of non-default cases:", total_non_default)
print(
    "This is equivalent to {:.2f}% of the total loans".format(
        (total_non_default / total_loans) * 100
    )
)
```

Number of default cases: 6463

This is equivalent to 21.94% of the total loans

Number of non-default cases: 22996

This is equivalent to 78.06% of the total loans

## 2.3 Split Dataset into Train and Test

```
In [11]: # creates the X and y data sets
X = df.drop("loan_status", axis=1).values
y = df["loan_status"].values

# splits into training and test set
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=2022, stratify=y
)
```

## 2.4 Balancing Data for Training

Since we saw that the data was highly imbalanced, we performed oversampling with synthetic minority over-sampling technique (SMOTE) with the package below.

```
In [12]: # counts the number of classes before oversampling
print("Before balancing:", Counter(y_train))

# defines the resampler
resampler = SMOTETomek(random_state=2022, n_jobs=-1)

# transforms the data set
X_balanced, y_balanced = resampler.fit_resample(X_train, y_train)

# counts the number of classes after oversampling
print("After balancing:", Counter(y_balanced))
```

Before balancing: Counter({0: 17247, 1: 4847})  
After balancing: Counter({1: 16988, 0: 16988})

```
In [13]: # plots before and after SMOTE
plt.figure(figsize=(12, 4))

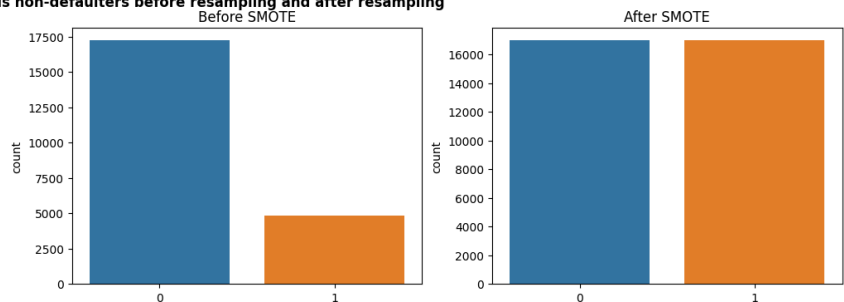
plt.subplot(1, 2, 1)
sns.countplot(x=y_train)
plt.title("Before SMOTE")

plt.subplot(1, 2, 2)
sns.countplot(x=y_balanced)
plt.title("After SMOTE")

plt.suptitle(
    "Fig. 2: Top: A plot of the defaulters versus non-defaulters before resampling",
    fontweight="bold",
    horizontalalignment="right",
)

plt.show()
```

Fig. 2: Top: A plot of the defaulters versus non-defaulters before resampling and after resampling



```
In [14]: print("Total records BEFORE:", X_train.shape[0])
print("Total records AFTER:", X_balanced.shape[0])
print("Difference =", X_train.shape[0] - X_balanced.shape[0])
```

Total records BEFORE: 22094  
Total records AFTER: 33976  
Difference = -11882

We now see that both the bad and good target variable are equal. Why do we need to balance our dataset?

## 2.5 Training the Models and Getting the Performance Metrics

As the process of model building affects the business, a discussion with business owners on the objective of the model will highly affect how we set thresholds. If the purpose is customer acquisition at the expense of risk, we can always lower the threshold of probability so as to get as many customers as possible. If we are interested in optimizing to get high profits or minimize high risks, then we can increase the threshold.

```
In [15]: # Set the threshold for defaults
THRESHOLD = 0.50
```

In this exercise, we will use two classifiers, logistic regression that is highly used by banks to discriminate between defaulters and non-defaulters, because it is easy to understand how it works.

```
In [16]: # list of classifiers
classifiers = [
    LogisticRegression(max_iter=220, random_state=2022),
    RandomForestClassifier(random_state=2022),
]
```

The function below trains the models on a dataset and evaluates their performance on unseen data.

```
In [17]: def calculate_model_metrics(model, X_test, y_test, model_probs, threshold):
    """
    Calculates Accuracy, F1-Score, PR AUC
    """
    # keeps probabilities for the positive outcome only
    probs = pd.DataFrame(model_probs[:, 1], columns=["prob"])

    # applies the threshold
    y_pred = probs["prob"].apply(lambda x: 1 if x > threshold else 0)

    # calculates f1-score
    f1 = f1_score(y_test, y_pred)

    # calculates accuracy
    accuracy = accuracy_score(y_test, y_pred)

    # calculates kappa score
    kappa = cohen_kappa_score(y_test, y_pred)

    # calculates AUC
    auc_score = roc_auc_score(y_test, probs)

    # calculates the precision
    precision = precision_score(y_test, y_pred)

    # calculates the recall
```

```

recall = recall_score(y_test, y_pred)

return accuracy, kappa, f1, auc_score, precision, recall

```

```

In [18]: def get_classifiers_performance(
X_train, X_test, y_train, y_test, threshold, classifiers
):
    # creates empty data frame
    df_performance = pd.DataFrame()

    for clf in classifiers:
        print("Training " + type(clf).__name__ + "...")
        # fits the classifier to training data
        clf.fit(X_train, y_train)

        # predict the probabilities
        clf_probs = clf.predict_proba(X_test)

        # calculates model metrics
        (
            clf_accuracy,
            clf_kappa,
            clf_f1,
            clf_auc,
            clf_precision,
            clf_recall,
        ) = calculate_model_metrics(clf, X_test, y_test, clf_probs, threshold)

        # creates a dict
        clf_dict = {
            "model": [type(clf).__name__, "---"],
            "precision": [clf_precision, np.nan],
            "recall": [clf_recall, np.nan],
            "f1-Score": [clf_f1, np.nan],
            "ROC AUC": [clf_auc, np.nan],
            "accuracy": [clf_accuracy, np.nan],
            "cohen kappa": [clf_kappa, np.nan],
        }

        # concatenate Data Frames
        df_performance = pd.concat([df_performance, pd.DataFrame(clf_dict)])

    # resets Data Frame index
    df_performance = df_performance.reset_index()

    # drops index
    df_performance.drop("index", axis=1, inplace=True)

    # gets only the odd numbered rows
    rows_to_drop = np.arange(1, len(classifiers) * 2, 2)

    # drops unwanted rows that have no data
    df_performance.drop(rows_to_drop, inplace=True)

    # returns performance summary
    return df_performance

```

```

In [19]: # calculates classifiers performance
df_performances = get_classifiers_performance(

```

```

X_balanced, X_test, y_balanced, y_test, THRESHOLD, classifiers
)
# highlight max values for each column
df_performances.style.highlight_max()

```

Training LogisticRegression...

Training RandomForestClassifier...

Out[19]:

|   | model                  | precision | recall   | f1-Score | ROC AUC  | accuracy | cohen kappa |
|---|------------------------|-----------|----------|----------|----------|----------|-------------|
| 0 | LogisticRegression     | 0.555309  | 0.773515 | 0.646496 | 0.871806 | 0.814392 | 0.525213    |
| 2 | RandomForestClassifier | 0.897513  | 0.737005 | 0.809378 | 0.927522 | 0.923829 | 0.762356    |

Clearly, the random forest classifier outperforms logistic regression in nearly all the metrics with the exception of recall.

## 2.6 Probability Distribution

```

In [20]: # instantiates the classifiers
lr_clf = LogisticRegression(max_iter=220, random_state=2022)
rf_clf = RandomForestClassifier(random_state=2022)

# trains the classifiers
lr_clf.fit(X_balanced, y_balanced)
rf_clf.fit(X_balanced, y_balanced)

# store the predicted probabilities for class 1
y_pred_lr_prob = lr_clf.predict_proba(X_test)[: , 1]
y_pred_rf_prob = rf_clf.predict_proba(X_test)[: , 1]

```

```

In [21]: # sets plot size
plt.figure(figsize=(10, 6))

# plots
sns.kdeplot(y_pred_lr_prob, label="Logistic Regression")
sns.kdeplot(y_pred_rf_prob, label="Random Forest")

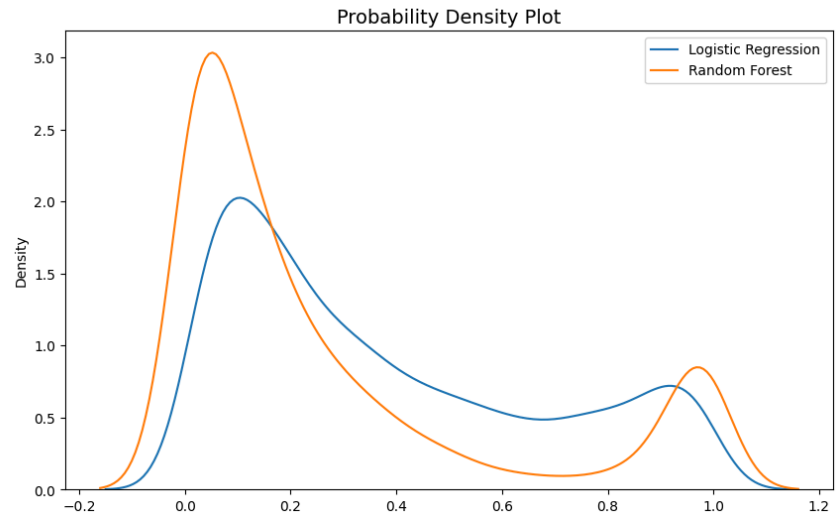
# sets the plot features
plt.title("Probability Density Plot", fontsize=14)
plt.legend()

plt.suptitle(
    "Fig. 3: Top: A probability density plot for the predicted performance of cu
    fontweight="bold",
    horizontalalignment="right",
)

# displays the plot
plt.show()

```

**Fig. 3: Top: A probability density plot for the predicted performance of customers**



We can observe in the density plot above that the largest concentration of probabilities is around 0, and logistic regression has moderate distributed probabilities.

```
In [22]: # set axes
fig, ax = plt.subplots()

# set fig size
ax.figure.set_size_inches(12, 8)

# plot probability - Logistic Regression
plt.subplot(2, 2, 1)
ax = stats.probplot(y_pred_lr_prob, plot=plt)

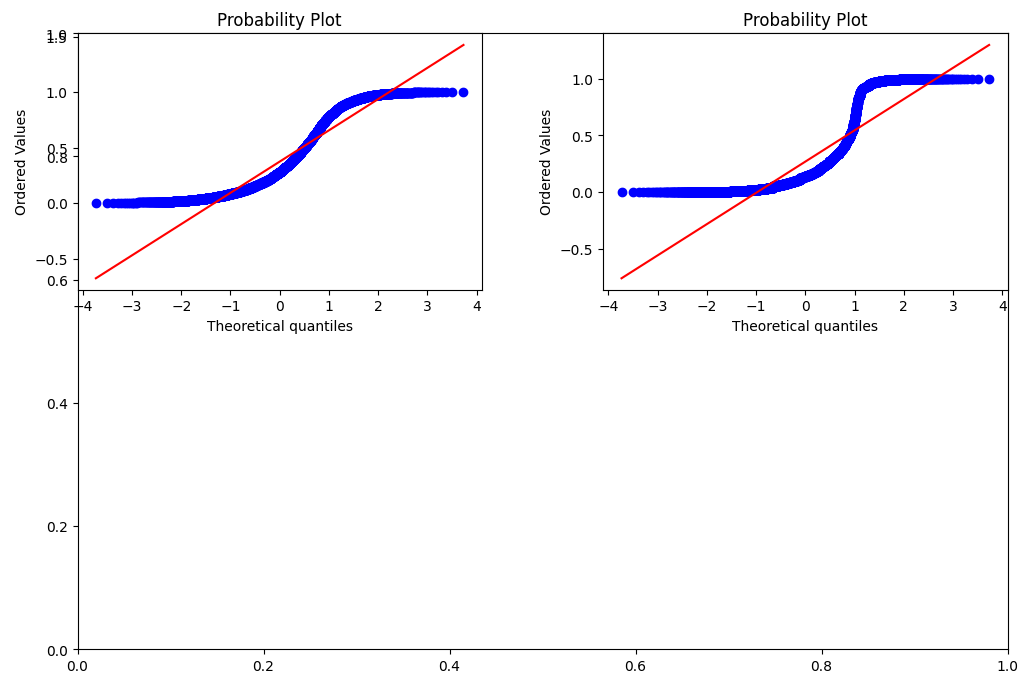
# plot probability - Random Forest
plt.subplot(2, 2, 2)
ax = stats.probplot(y_pred_rf_prob, plot=plt)

plt.subplots_adjust(wspace=0.3)
plt.subplots_adjust(hspace=0.4)

plt.suptitle(
    "Fig. 4: Top: Comparing model performance and theoretical expectation",
    fontweight="bold",
    horizontalalignment="right",
)

# displays the plot
plt.show()
```

**Fig. 4: Top: Comparing model performance and theoretical expectation**



The data seem to follow the theoretical distribution when we use the two models.

```
In [23]: # calculates the Brier Score Loss
bsl_lr = brier_score_loss(y_test, y_pred_lr_prob, pos_label=1)
bsl_rf = brier_score_loss(y_test, y_pred_rf_prob, pos_label=1)

# prints the calculated Brier Score Loss for each algorithm probability
print(f"Brier Score Loss (Logistic Regression): {np.round(bsl_lr, 2)}")
print(f"Brier Score Loss (Random Forest): {np.round(bsl_rf, 2)}")
```

Brier Score Loss (Logistic Regression): 0.14

Brier Score Loss (Random Forest): 0.07

The smaller the value of the Brier score, the better. The Brier score is made up of refinement loss and calibration loss. Clearly, we can see that random forest performs better since it give us a lower Brier score.

```
In [24]: # makes predictions
y_pred_lr = lr_clf.predict(X_test)
y_pred_rf = rf_clf.predict(X_test)

print("Classification Report for " + type(lr_clf).__name__)
print(classification_report(y_test, y_pred_lr))

print("\nClassification Report for " + type(rf_clf).__name__)
print(classification_report(y_test, y_pred_rf))
```

| Classification Report for LogisticRegression |           |        |          |         |  |
|----------------------------------------------|-----------|--------|----------|---------|--|
|                                              | precision | recall | f1-score | support |  |
| 0                                            | 0.93      | 0.83   | 0.87     | 5749    |  |
| 1                                            | 0.56      | 0.77   | 0.65     | 1616    |  |
| accuracy                                     |           |        | 0.81     | 7365    |  |
| macro avg                                    | 0.74      | 0.80   | 0.76     | 7365    |  |
| weighted avg                                 | 0.85      | 0.81   | 0.82     | 7365    |  |

| Classification Report for RandomForestClassifier |           |        |          |         |  |
|--------------------------------------------------|-----------|--------|----------|---------|--|
|                                                  | precision | recall | f1-score | support |  |
| 0                                                | 0.93      | 0.98   | 0.95     | 5749    |  |
| 1                                                | 0.90      | 0.74   | 0.81     | 1616    |  |
| accuracy                                         |           |        | 0.92     | 7365    |  |
| macro avg                                        | 0.91      | 0.86   | 0.88     | 7365    |  |
| weighted avg                                     | 0.92      | 0.92   | 0.92     | 7365    |  |

We observe that random forest performs better than logistic regression on unseen data. You are tasked to plot the receiver operation curve and the confusion matrix of the two models.

In the next step we will train deep learning models to estimate probability of default.

## Deep Learning

We start by importing `keras` classifier to run the model.

```
In [25]: # !pip install tensorflow
# !pip install scikeras
```

```
In [26]: import logging

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense, Dropout
# from tensorflow.keras.wrappers.scikit_learn import KerasClassifier
from scikeras.wrappers import KerasClassifier, KerasRegressor

tf.get_logger().setLevel(logging.ERROR)
```

```

2025-10-15 13:16:05.395032: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.
2025-10-15 13:16:05.395533: I tensorflow/core/util/port.cc:153] oneDNN custom ope
rations are on. You may see slightly different numerical results due to floating-
point round-off errors from different computation orders. To turn them off, set t
he environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-10-15 13:16:05.444465: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in performa
nce-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512
_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild Tens
orFlow with the appropriate compiler flags.
2025-10-15 13:16:06.683833: I tensorflow/core/util/port.cc:153] oneDNN custom ope
rations are on. You may see slightly different numerical results due to floating-
point round-off errors from different computation orders. To turn them off, set t
he environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-10-15 13:16:06.684609: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.

```

We then write a function with the model layers and activation functions at each layer

```

In [27]: def model_function(dropout_rate, verbose=0):
# Select a `keras` model, sequential function allows us to specify our neural
model = keras.Sequential()
# First Layer is dense implying that the features are connected to every sin
model.add(Dense(128, kernel_initializer="normal", activation="relu", input_dim=100))
model.add(Dense(64, kernel_initializer="normal", activation="relu"))
model.add(Dense(8, kernel_initializer="normal", activation="relu"))
# Drop out is added to ignore irrelevant neurons
model.add(Dropout(dropout_rate))
# We add sigmoid to classify the target
model.add(Dense(1, activation="sigmoid"))
# the loss function is binary cross entropy because the problem is a binary
model.compile(loss="binary_crossentropy", optimizer="rmsprop")
return model

```

The next cell defines the model parameters and hyperparameters, train the model and validate the model on test dataset.

```

In [28]: model = KerasClassifier(
    build_fn=model_function, dropout_rate=0.2, verbose=0, batch_size=50, epochs=100
)
model.fit(X_train, y_train)
y_predict_dl = model.predict(X_test)

```

```

/usr/local/lib/python3.11/site-packages/scikeras/wrappers.py:925: UserWarning: ``
build_fn`` will be renamed to ``model`` in a future release, at which point use o
f ``build_fn`` will raise an Error instead.
  X, y = self._initialize(X, y)
/usr/local/lib/python3.11/site-packages/keras/src/layers/core/dense.py:92: UserWa
rning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using S
equential models, prefer using an `Input(shape)` object as the first layer in the
model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2025-10-15 13:16:06.873891: E external/local_xla/xla/stream_executor/cuda/cuda_pl
atform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit:
UNKNOWN ERROR (303)

```

We now evaluate the model using several metrics

```
In [29]: ROC_AUC_metric = roc_auc_score(y_test, pd.DataFrame(y_predict_dl.flatten()))
print("Deep Learning ROC AUC is {:.4f}".format(ROC_AUC_metric))
```

Deep Learning ROC AUC is 0.8366

```
In [30]: print("\nClassification Report for deep learning model")
print(classification_report(y_test, y_predict_dl.flatten()))
```

Classification Report for deep learning model

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.92      | 0.98   | 0.95     | 5749    |
| 1            | 0.92      | 0.69   | 0.79     | 1616    |
| accuracy     |           |        | 0.92     | 7365    |
| macro avg    | 0.92      | 0.84   | 0.87     | 7365    |
| weighted avg | 0.92      | 0.92   | 0.91     | 7365    |

```
In [31]: print("\nThe brier score is equal to ")
brier_score_loss(y_test, y_predict_dl.flatten(), pos_label=1)
```

The brier score is equal to

Out[31]: 0.08119484046164291

We can observe that the model performance is comparable to random forest model without much data preparation or data balancing.

The next step is to help a business make the right decision. The discussion for this is beyond the scope of this lesson and we will therefore only give a brief discussion of what to expect.

## 2.7 Business Decision Making

The final step of modeling is to evaluate if our modeling met the set Key Performance Indicators (KPIs) by the institution. Among the metrics to measure are:

### 2.7.1 Acceptance Rate

The threshold for an acceptance rate is used for setting the percentage of new loans business are willing to disburse. We use test data as a proxy for fresh new loans to measure the performance.

We use the threshold to assign new `loan_status` values. We then observe if the distribution of the bads and goods changes.

### 2.7.2 Bad Rates

Now that we have the acceptance rate, we do the analysis on bad rate percentages within the loans that met the threshold. This allows us to see the percentage of defaults that our model will accept.

In summary, we set an acceptance rate so as to have fewer bad loans in our portfolio.

### 2.7.3 Strategy

At this point, a strategy meeting with project (business) owners is called where we brainstorm with them and strategize on the best approach for using the scorecard. This is done by building a strategy table that has simulations with possible outcomes.

### 2.7.4 Total Loss

Finally, we estimate the total expected loss having concrete decisions from the data science team and input from business owners. We define the expected loss as the amount of money a financial lender might lose by lending to a borrower. The components of expected loss (EL) are:

- Probabilities of default (**PD**)
- The loss given default (**LGD**)
- The `loan_amnt`, which will be assumed to be the exposure at default (**EAD**).

The factors that contribute to expected loss are categorized into:

1. Borrower specific factors.
2. Economic environment.

The expected loss is calculated as shown below:

$$\text{Total Expected Loss} = \sum_{x=1}^n PD_x * LGD_x * EAD_x$$

$$\text{expected\_loss} = \text{prob\_default} * \text{lgd} * \text{loan\_amnt}$$

Probability of default as defined earlier is the inability of the borrower to repay their debt in full or on time/ schedule.

Loss given default on the other hand is the proportion of the borrowed amount by the customer that the financial institution will be unable to recover once the customer defaults.

Exposure at default is the total sum that a lender is exposed to at the time of default.

Let us consider an example below.

Consider the case of a mortgage worth \$1,000,000 and the lender is willing to finance 80% of the mortgage value. The borrower pays back \$100,000 and our model shows that the probability of default is 20%. Further assume that the bank can recover some of the money by auctioning the house at \$560,000 after the customer has defaulted. Then, this is how we will calculate the expected loss:

- Cost of House = \$1,000,000
- Lender finances 80% of the cost, that is, loan amount = \$800,000
- If the borrower pays back \$100,000 then exposure at default = \$800,000 - \$100,000 = \$700,000
- Probability of default = 0.2

- Loss given default =  $\frac{\$700,000 - \$560,000}{\$700,000} = 20\%$

So the expected loss is:  $\text{PD} \times \text{LGD} \times \text{EAD} = 0.2 \times 0.2 \times \$700,000 = \$28,000$

In this lesson, we have covered an application of machine learning in credit risk management. Can you think of how the same can be implemented using a deep learning algorithm?

### 3. Conclusion

This module introduced the concept of deep learning and its application in finance. In the next module, we will study hyperparameter tuning in machine learning models to improve their performance.

#### References

1. Cooper, Michael James. *A Deep Learning Prediction Model for Mortgage Default*. 2018. University of Bristol, Masters thesis.
2. Giesecke, Kay. "Credit Risk Modeling and Valuation: An Introduction." 2004. Available at SSRN 479323.
3. Sirignano, Justin et al. "Deep Learning for Mortgage Risk." *arXiv preprint arXiv:1607.02470*. 2016.

---

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.